

Documentación Frontend Completa — Eventos DSW

El frontend del proyecto Eventos DSW fue desarrollado utilizando React + Vite + TypeScript. La aplicación consume eventos desde un backend Express/Prisma y utiliza una arquitectura basada en componentes reutilizables.

Arquitectura del proyecto

```
src
├── Components
│   ├── events
│   │   ├── EventCard.tsx
│   │   └── EventGrid.tsx
│   ├── home
│   │   └── HeroSection.tsx
│   ├── layout
│   │   ├── Navbar.tsx
│   │   ├── Footer.tsx
│   │   └── MainLayout.tsx
├── Pages
│   ├── Home
│   │   └── Homepage.tsx
│   ├── EventPage
│   │   └── EventPage.tsx
│   ├── CreateEventPage
│   └── ProfilePage
├── routes
│   └── AppRoutes.tsx
├── services
│   ├── api.ts
│   └── eventService.ts
├── styles.css
└── main.tsx
```

Configuración TailwindCSS

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    './index.html',
    './src/**/*.{js,ts,jsx,tsx}',
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}

@tailwind base;
```

```

@tailwind components;
@tailwind utilities;

body {
  font-family: system-ui, Arial, sans-serif;
  margin: 0;
}

```

Configuración Axios

```

import axios from "axios";

export const api = axios.create({
  baseURL: "http://localhost:3001/api/v1",
});

```

Servicio de eventos

```

import { api } from "../api";

export async function getEvents() {
  const response = await api.get("/eventos");

  return response.data.data;
}

```

Navbar

```

import { Link } from "react-router-dom";

export default function Navbar() {
  return (
    <nav className="bg-white border-b sticky top-0 z-50">
      <div className="max-w-7xl mx-auto px-6 py-4 flex items-center justify-between">
        <h1 className="text-2xl font-bold">
          Cartelera
        </h1>

        <div className="flex items-center gap-6">
          <Link to="/">Inicio</Link>

          <Link to="/crear-evento">
            Crear Evento
          </Link>

          <Link to="/perfil">
            Perfil
          </Link>
        </div>
      </div>
    </nav>
  );
}

```

HeroSection

```

export default function HeroSection() {
  return (
    <section
      className="
        bg-gradient-to-r
        from-violet-600

```

```

        to-pink-500
        rounded-[32px]
        p-12
        text-white
        mb-10
      "
    >
    <h1 className="text-6xl font-bold mb-6">
      Eventos locales,
      cultura viva
    </h1>

    <input
      type="text"
      placeholder="Buscar eventos..."
      className="
        w-full
        max-w-xl
        px-6
        py-4
        rounded-2xl
        text-black
      "
    />
  </section>
);
}

```

EventGrid

```

import EventCard from "../EventCard";

type Event = {
  id: string;
  titulo: string;
  descripcion?: string;
};

type Props = {
  events: Event[];
};

export default function EventGrid({
  events,
}: Props) {
  return (
    <div
      className="
        grid
        grid-cols-1
        md:grid-cols-2
        lg:grid-cols-3
        gap-6
      "
    >
      {events.map((event) => (
        <EventCard
          key={event.id}
          event={event}
        />
      ))}
    </div>
  );
}

```

Homepage

```

import { useQuery } from "@tanstack/react-query";

import MainLayout from "../../Components/layout/MainLayout";

import EventGrid from "../../Components/events/EventGrid";

import HeroSection from "../../Components/home/HeroSection";

import { getEvents } from "../../services/eventService";

export default function HomePage() {
  const {
    data,
    isLoading,
    error,
  } = useQuery({
    queryKey: ["events"],
    queryFn: getEvents,
  });

  return (
    <MainLayout>
      <HeroSection />

      {isLoading && (
        <p>Cargando eventos...</p>
      )}

      {data && <EventGrid events={data} />}
    </MainLayout>
  );
}

```

React Query

React Query fue configurado para manejar: - fetching de eventos, - estados de carga, - manejo de errores, - cache automático.

Objetivos visuales

La interfaz se diseñó tomando inspiración en plataformas modernas de eventos. Se priorizó: - uso de gradientes, - cards visuales, - mucho espacio visual, - layout responsive, - diseño minimalista.

Próximos pasos

- Implementar EventPage. - Crear formulario de creación de eventos. - Agregar filtros reales. - Integrar búsqueda. - Mejorar responsive mobile. - Agregar imágenes reales.